

Program 4:

Integrate Git with Docker for source-controlled application build

07/11/2025

Project Directory:

```
prog4
|- Dockerfile
|- app.py
|- requirements.txt
```

Program code and steps for execution:

Step 1: Create a folder with the name prog4 for the program 4 execution

```
> mkdir prog4
```

```
> cd prog4/
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~$ cd devops_lab/
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab$ mkdir prog4
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab$ cd prog4/
```

Step 2: Create the three files

1. app.py 2. Dockerfile 3. requirements.txt

And the below images shows the content to be included in each file

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog4$ touch app.py requirements.txt Dockerfile
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog4$ ls
app.py  Dockerfile  requirements.txt
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog4$ nano app.py
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog4$ cat app.py
from flask import Flask
app = Flask(__name__)
```

```
@app.route('/')
def home():
    return "Program 4: Flask App running inside Docker!"
```

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog4$ nano requirements.txt
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog4$ cat requirements.txt
Flask==2.2.5
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog4$ nano Dockerfile
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog4$ cat Dockerfile
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY app.py .
```

```
EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

Step 3: set the Git name and email first, then initialize

```
> git config --global user.name "<username>"
> git config --global user.email "<user-email>"
> git init
> git add .
> git commit -m "Initial commit for Program 4"
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog4$ git config --global user.name "Chinmayi-Aradhya"
git config --global user.email "chinmayiaradhya03@gmail.com"
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog4$ git init
git add .
git commit -m "Initial commit for Program 4"
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/chinnu/devops_lab/prog4/.git/
[master (root-commit) f6f4fdf] Initial commit for Program 4
3 files changed, 22 insertions(+)
create mode 100644 Dockerfile
create mode 100644 app.py
```

Step 4: Login to your github account from browser and create a new repository with the name prog4

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk ().*

1

General

Owner *

Chinmayi-Aradhya

Repository name *

prog4

✔ prog4 is available.

Great repository names are short and memorable. How about [fantastic-enigma?](#)

Description

Program 4

9 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

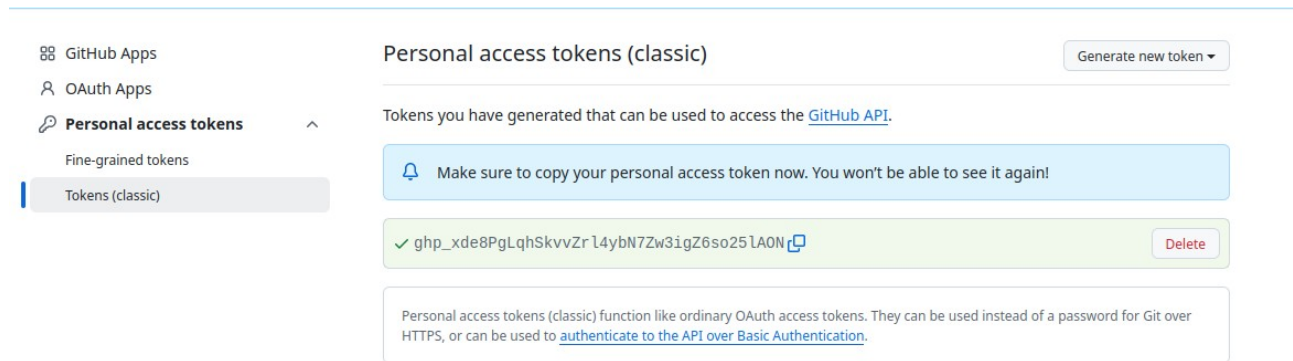
Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

Step 5: In the github page follow the below order

> login > settings > developer settings > Personal Access Tokens > Token (classic) > Generate new token > copy the token



The screenshot shows the GitHub 'Personal access tokens (classic)' page. On the left, a sidebar lists 'GitHub Apps', 'OAuth Apps', 'Personal access tokens' (selected), 'Fine-grained tokens', and 'Tokens (classic)'. The main content area has a 'Generate new token' button. Below it, a message says 'Tokens you have generated that can be used to access the [GitHub API](#).' A blue alert box states: 'Make sure to copy your personal access token now. You won't be able to see it again!'. A green box shows a token: 'ghp_xde8PgLqhSkvvZr l4ybN7Zw3igZ6so25 lA0N' with a copy icon and a 'Delete' button. At the bottom, a note explains that classic tokens function like ordinary OAuth access tokens and can be used instead of a password for Git over HTTPS, or to authenticate to the API over Basic Authentication.

Step 6: Push the project to GitHub using the following commands,

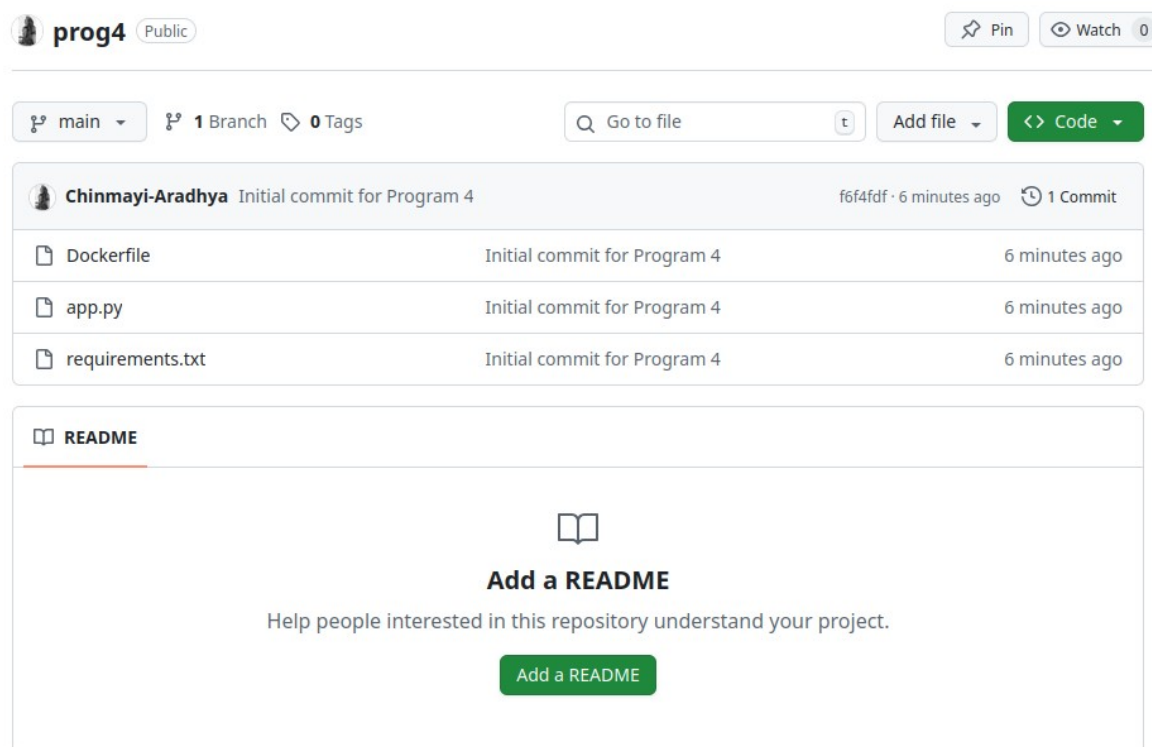
> git remote add origin https://github.com/<your-username>/prog4.git

> git branch -M main

> git push -u origin main

Here after the above commands it will ask the user name and password then paste the token which we have created

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/p
rog4$ git remote add origin https://github.com/Chinmayi-Aradhya/prog4.git
git branch -M main
git push -u origin main
Username for 'https://github.com': Chinmayi-Aradhya
Password for 'https://Chinmayi-Aradhya@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 621 bytes | 621.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Chinmayi-Aradhya/prog4.git
```



The screenshot shows the GitHub repository page for 'prog4' (Public). It includes a 'Pin' button and a 'Watch' button (0). The repository has 1 branch (main) and 0 tags. A search bar and 'Add file' button are present. The commit history shows an initial commit for 'Program 4' by Chinmayi-Aradhya, 6 minutes ago, with 1 commit. The commit includes files: Dockerfile, app.py, and requirements.txt. Below the commit list, there is a 'README' section with a placeholder for a README file, an 'Add a README' button, and a message: 'Help people interested in this repository understand your project.'